

cs-mail Product Build Plan

Server, Administration, Member Desktop, and Sender Request Page

C-SQD

Draft 0.5 — 25 September 2026

Abstract

This plan turns the implemented Rust domain into a runnable product. It builds the core relationship-request loop end to end first—operator service, identity enrollment, request, decision, and settlement, driven from a command-line client—and then adds depth: a public request page for senders without an account, the macOS member application, recovery across time and failure, the annual service and member-cycle experience, and a small pilot. *cs-mail*’s core is also required by the future Phoros Account, so user testing informs design rather than deciding whether development continues. The pilot introduces no real redeemable value.

Design principles. Before changing the implementation or this design, read the [Rust-domain design principles](#) (`docs/rust-domain-principles.md`). They are binding for *cs-mail* and *identity-model*; new work must not regress them.

1 What Changed From Draft 0.4

Draft 0.4 built complete layers in sequence and reached the first relationship request only at its third and fourth milestones. This draft reorders the work:

- The core request loop runs end to end from a developer CLI at Milestone 2, before any graphical client.
- Enrollment and annual service use the existing *identity-model* and billing domains from the start, with simulated evidence supplied by development support code. No fixture or bypass path is later replaced.
- A public *sender request page* lets a person without a *cs-mail* account pick a request class, pay, and write a message. It first requires a guest-sender domain design (Milestone 3a).
- The macOS application follows the proven loop. Billing and distribution *experience* moves later; its domain rules are already implemented.
- Each milestone names the conjectures its tests challenge, following principle 10 of the [Rust-domain principles](#), plus a learning question that does not gate progress.
- Development and testing are local. There is no hosted CI.

2 Build on the Implemented Domain Contracts

The *Protocol Specification* governs requests and permission. The normative C-SQD financial-program section of the *Deployment and Migration Profile* governs annual utility billing, the pool, and equal

member distributions. The *Rust Reference Architecture* describes the current ownership boundaries. Start with the [C-SQD domain model](#) for confirmed rules and deferred choices, the [product identity model](#) for accounts and logins, and the [Rust-domain principles](#) for how new code is shaped and validated.

The workspace already implements the request lifecycle with recipient-defined request classes, lane acceptance with atomic refund settlement, native correspondence, consent-scoped decryption, request-specific payment operations and refunds, forfeiture maturity, annual allocation, fixed service contracts with advance collection, funded coverage, and automatic lump-sum distribution preparation. Signed service and client APIs, PostgreSQL transactions, and worker libraries exercise those rules with a simulated external provider. They are not yet a running server or a user-facing application.

Product work reuses those owners and APIs. It does not recreate deprecated labels, aliases, fallback readers, or parallel implementations. When product work shows that a domain rule is wrong or incomplete, the change is made as an explicit, reviewed cutover in the owning domain, not patched in an adapter.

3 The Product and the Pilot

Clients and their roles.

- **Member application (macOS, Tauri).** Holds endpoint keys, the encrypted inbox, request decisions, request classes, lanes, blocks, and financial status. Native content is end-to-end encrypted.
- **Sender request page (web).** Lets a person without a cs-mail account select a recipient's published request class, authorize one conditional charge, and write a message. The page is served by the provider, so it cannot carry the end-to-end promise; its messages are labeled as an explicit privacy downgrade, like inbound SMTP.
- **Developer CLI.** Drives member journeys before the desktop application exists and remains a development and operations tool.
- **Administration CLI.** An authenticated operator client.

A full web inbox for members is not planned. It would place endpoint keys in code delivered by the provider on each page load, which conflicts with the promise that the provider cannot decrypt content. The client core compiles to WebAssembly, so the option remains available if that trust model is ever accepted or mitigated.

Pilot. An invite-only group served by one provider, one region, and one PostgreSQL deployment. Recipients are recruited from people who receive substantial unsolicited contact; senders are people trying to reach them, including guests once the request page ships.

Value. A deterministic external-payment simulator approves, declines, delays, or reports an uncertain charge; confirms or fails a refund; and executes or fails a member payment. All amounts are explicitly nonredeemable. Users receive no spendable test-credit wallet; the operator configures simulator scenarios.

Out of scope for the pilot. Independent-provider federation, a full SMTP relay, public signup, live payments, mobile releases, and multi-region operation.

4 Architecture of the Product Layer

```
crates/
  existing cs-mail-* crates      # domain, application, storage, workers
  cs-mail-client                 # extended client core (ports for keys,
                                #   local storage, transport, clock)
  cs-mail-api-types              # add when transport contracts need it
  cs-mail-mobile-ffi             # Milestone 8
apps/
  cs-maild                      # HTTP API, workers, sender page hosting
  cs-mail-admin                  # authenticated operator CLI
  cs-mail-cli                    # developer member client
  cs-mail-desktop                # Tauri shell and macOS adapters
  cs-mail-web                    # sender request page (static assets)
ui/                              # interface components shared by
                                #   desktop and web
../identity-model               # separate repository, sibling checkout
```

Dependencies point inward: applications use the client core and application layer, which use the domain; domain crates never depend on applications. **Applications are adapters.** They validate input, invoke application use cases, and render outcomes. They make no domain decisions, obtain trusted time only as the application-persistence boundary prescribes, and never translate a signed command into a different meaning. The client core compiles to WebAssembly; the member application nonetheless runs it natively through Tauri.

identity-model remains a separate repository because it will also serve the Phoros Account. cs-mail pins the reviewed identity sources in `identity-source.sha256`; commit identity-model changes before the cs-mail changes that depend on them and keep the manifest in step.

5 Contracts That Apply to Every Milestone

Keys and authority. Endpoint private keys use operating-system secure storage (or, for the developer CLI only, a local-file implementation of the same key-custody port) and never enter logs, configuration, SQLite, provider APIs, or diagnostics. Device enrollment and revocation require authenticated authority. Administrative credentials are distinct from user credentials; the admin CLI uses an authenticated API, never direct table writes.

Evidence and simulation. Simulated login, bank-ownership, funding, and payment evidence, together with scenario builders, live in development support code. They enter the product through the same ports as real evidence and are never presented as business commitments. Rates, thresholds, prices, and dates use explicitly versioned simulation policies.

Outcomes and uncertainty. Every visible outcome has a durable cause and a stable retry identity. “Outcome unknown,” “failed,” “expired,” and “denied” remain distinct through storage, API, and interface. The interface separates submission preparation, awaiting a decision, refund due, refund completed, annual service coverage, finalized member allocation, and payment completion. A distribution never marks a service year unpaid. No screen suggests that rejecting a sender pays the recipient or that a payable can fund a request.

Versions and data. Protocol, wire, API, event, policy, and database versions are separate. Until the fresh-database policy ends (decided at Milestone 0), schema changes may replace earlier migrations; afterwards migrations are forward-only. Logs contain scoped references and typed errors, not plaintext, secrets, identity mappings, full signed commands, or complete financial snapshots. Retention policy is explicit before pilot data exists.

Validation. Tests challenge named conjectures whose falsification would matter; each milestone lists its claims and agrees its testing scope before work begins. Claims about locking, atomicity, or concurrency are challenged against PostgreSQL, not an in-memory adapter. Guarantees already enforced by types need no redundant runtime test. Demonstration scripts show a journey; they are not evidence for a claim. At each handoff, run the local quality gate:

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace
CS_MAIL_TEST_DATABASE_URL=... cargo test --workspace -- --ignored
```

6 Milestone Map

Stage	Result	Exit evidence
0	Product skeleton	Buildable apps, inward dependencies, quality gate
1	Service and identity boot	Clean start, restart, invites, simulator scenarios
2	Core loop from the CLI	Every request outcome, from a clean database
3a	Guest-sender domain design	Agreed types, evidence, ownership, and rules
3b	Sender request page	A guest completes a request; every outcome settles
4	macOS member application	Milestone 2 journeys in the app, keys in keychain
5	Time and failure	Crash, race, and deadline claims challenged
6	Annual service and member cycle	A full simulated year, including nonrenewal
7	Pilot	Signed builds, recovery, restore, pilot findings
8	Portability and Phoros	Windows, Swift/Kotlin parity, Phoros integration

Each stage depends on the preceding one, except that Milestone 4 may proceed before 3b if the guest-sender design remains unresolved. A milestone is complete when its demonstration runs from a clean checkout and its claims have been challenged, not merely when new code compiles.

7 Milestone 0: Product Skeleton

Create `apps/cs-maild`, `apps/cs-mail-admin`, and `apps/cs-mail-cli` with shared package meta-data, lints, and dependency versions. Each reports its version and help. Add a local script that runs the full quality gate without hiding individual checks, and document the inward dependency rule.

Decision to record: when the fresh-database policy ends and migrations become forward-only—at the latest, before any pilot data is created.

Exit: clean checkout, passing gate, running binaries. No listener, enrollment, or domain change belongs to this stage.

8 Milestone 1: The Service and Identity Boot

The operator can migrate an empty database, bootstrap one provider idempotently, start the API, create invites, publish service offers and request pricing, and select simulator scenarios—without editing SQL. Use Tokio, Axum, Rustls, structured tracing, and Clap. Keep synchronous storage behind a bounded blocking boundary until a tested replacement is warranted. Readiness distinguishes process health from database readiness.

The identity-model enrollment service runs beside `cs-maild`, with a development OIDC provider (the Keycloak realm in `identity-model/ci`) and simulated bank-ownership evidence from development support code. The payment simulator persists operation identities and supports delayed, duplicate, out-of-order, and failed results; its scenario changes are audited and separate from canonical financial writes.

Claim (PostgreSQL).

Repeating bootstrap cannot create a second provider authority or replace the key registry.

Falsified by: Two bootstrap runs, including one interrupted midway, leave two authorities or a changed registry.

Matters because: Duplicate authority would let two keys speak for the provider.

Claim (PostgreSQL).

Replaying or concurrently redeeming one invite cannot enroll two accounts.

Falsified by: Two concurrent redemptions both succeed.

Matters because: One account per person is an anti-abuse objective and a member-share rule.

Claim (PostgreSQL).

A restart cannot duplicate a simulated payment operation.

Falsified by: A killed and restarted service shows two operations for one operation identity.

Matters because: Duplicate operations become duplicate charges once payments are real.

Exit: a clean-start transcript, restart demonstration, and redacted diagnostics.

9 Milestone 2: The Core Loop From the Command Line

Alice and Bob enroll through identity-model using the developer CLI and establish their protocol identities and device keys. Each purchases annual service through the existing billing domain; the simulator finalizes funding and coverage begins within the purchased interval.

Bob publishes request classes. Alice resolves Bob, reviews C , S , the total conditional charge, the decision deadline, and possible refunds; authorizes one charge; and submits an encrypted initial message after funding reaches finality. Bob accepts with standing permission or an express lane, rejects, blocks, or lets the request expire. Alice sends follow-ups when allowed. Both sides see settlement: refund owed and confirmed, processing revenue retained, collateral pending forfeiture. Uncertain outcomes are shown as uncertain.

Claim (PostgreSQL).

Retrying a submission whose outcome is unknown cannot create a second request or charge.

Falsified by: A client that loses the response and retries produces two requests or two captures.

Matters because: One request must remain one charge.

Claim (PostgreSQL).

Acceptance commits permission and the full refund obligation together or not at all.

Falsified by: A failure injected during acceptance leaves a lane or permission without its refund obligation, or the reverse.

Matters because: Either half alone breaks the settlement contract.

Claim (in-memory and PostgreSQL).

An altered retry under an existing operation identity is rejected.

Falsified by: A retry with changed terms or content under the same identity is accepted.

Matters because: Retry identities would stop protecting request terms.

Exit: a scripted two-member demonstration of every outcome from a clean database. **Learning:** does the decision flow read clearly, and which wording confuses?

10 Milestone 3a: Guest-Sender Domain Design

The current domain recognizes two senders: members, with an account, verified bank association, and protocol identity; and organizational legacy senders, identified by a DMARC-authenticated domain. A person using the request page without an account is neither. Before any request-page code, record in the domain model and a design note:

- the guest sender as a distinct type, and what evidence (for example a verified email address or payment evidence) identifies one and what that evidence establishes;
- who owns guest eligibility and anti-abuse history, given that principal-recipient history is keyed on a principal a guest does not have;
- what acceptance grants a guest—permission bound to verified evidence, a path to membership, or a bounded lane;
- how refunds reach a guest’s payment method without an account;
- how guest messages are marked as non-end-to-end and how that label reaches the recipient.

Follow the existing treatment of legacy senders where it fits. This is the one planned change to the core domain; it is made as a reviewed cutover, not an adapter-level extension. **Exit:** the agreed design, with its conjectures stated. If it remains unresolved, Milestone 4 proceeds first.

11 Milestone 3b: Sender Request Page

Each member has a public request link. A guest opens it, selects a published class, reviews *C*, *S*, the refund rules and the privacy label, authorizes one simulated charge, and writes a message. Encryption to the recipient’s key on arrival is an application use case; plaintext is not stored or logged. The recipient decides from the developer CLI until Milestone 4. The guest receives outcome and refund notices by email.

Optionally, unknown senders emailing a member’s address receive an automatic reply with the request link, using `cs-mail-smtp-gateway` for sender authentication. This is in scope only if relay operations stay small.

Claim (PostgreSQL).

A guest cannot exceed pending-request limits for one recipient by changing email addresses.

Falsified by: Repeated requests under fresh addresses from the same payment evidence all reach the recipient. The strength of this claim is set by 3a.

Matters because: The request page would become a spam channel.

Claim (inspection and integration).

Guest-message plaintext is never written to storage, logs, or diagnostics.

Falsified by: A secret-scan of the database, logs, and diagnostics after a guest request finds the message text.

Matters because: The downgrade label would understate the exposure.

Exit: a guest completes a request end to end and every outcome settles. **Learning:** do strangers understand the page, and what collateral feels reasonable?

12 Milestone 4: macOS Member Application

Alice installs the application, enrolls or recovers through identity-model, and reopens the same account after quitting. Keys live in the macOS keychain behind the key-custody port; SQLite holds ciphertext, key references, cursors, minimized relationship metadata, and nonsecret preferences only. Portable client use cases coordinate enrollment and device management; the Tauri shell stays thin.

Bob sees the chosen class, one request with its grouped messages, and a clear deadline, and decides with permission explained first and money second. He manages request classes with collateral from the operator's bounded menu, follow-up preferences, lanes, and blocks. Financial status shows request charges and refunds separately from annual service and distributions. Interface components are shared with the request page.

Claim (integration).

Secret key material never reaches SQLite, logs, or diagnostics.

Falsified by: A secret-scan after enrollment, messaging, and export finds key bytes.

Matters because: The endpoint-encryption promise would be void.

Claim (integration).

After quit, reopen, and projection rebuild, every displayed outcome matches the server's canonical record.

Falsified by: A decision, refund, or uncertain charge appears differently after reopening.

Matters because: Users would act on false financial or permission state.

Exit: Milestone 2's journeys in the application, including keychain denial and quit/reopen. **Learning:** do recipients publish classes, how many, and how long do decisions take?

13 Milestone 5: Recover Across Time and Failure

Run scheduled deadlines, outbox delivery, payment reconciliation, and retention through supervised leased workers with a distinct scheduler authority, bounded retries, graceful shutdown, database TLS, and a bounded connection pool. Jobs recheck current state and use stable operation identities. Offline recipients and cursor replay must not duplicate delivery or financial effects.

Claim (PostgreSQL, crash injection).

A crash between capture and recording its response cannot lose or duplicate the charge.

Falsified by: After restart, reconciliation shows no capture or two captures for one request.

Matters because: Money would be lost or taken twice.

Claim (PostgreSQL, race).

Cancellation racing a delayed funding confirmation returns captured money exactly once.

Falsified by: The request is both cancelled and submitted, or the refund is issued twice or never.

Matters because: Cancellation is the user's safety valve before submission.

Claim (PostgreSQL).

Out-of-order provider events cannot reopen a terminal settlement.

Falsified by: A late event changes a settled request's obligations.

Matters because: Terminal financial outcomes must stay fixed.

Claim (PostgreSQL).

Coverage expiring while a request is unsettled stops new communication but not decision or settlement.

Falsified by: The recipient cannot decide, or a refund stalls, after coverage ends.

Matters because: Obligations outlive service coverage.

Decision to record: the bounded-acceptance timing edge cases currently deferred in the domain model.

14 Milestone 6: Annual Service and Member-Cycle Experience

Build the billing and distribution experience and run collection retries in operation. A member reviews a published offer with annual price, advance collection date, and fixed service interval; failed and late collection retries preserve the price and interval; a pending charge does not imply renewed service.

Finalize a simulated UTC calendar year from fixed funding and eligibility inputs, showing the once-only corporate assessment, equal minor-unit allocations, and restricted remainder. Pay each member one lump sum of the previous year's share to the associated verified bank account. With $U = \$120$, allocation scenarios of \$80, \$120, and \$170 each produce one payment, reported as rebate/excess of \$80/\$0, \$120/\$0, and \$120/\$50. Repeat with nonrenewal and closure, preserving the minimal payment route.

Claim (PostgreSQL).

A failed or unanswered payout attempt cannot create a second allocation or payable.

Falsified by: After failure and restart, a member has two payables for one pool year.

Matters because: Pool funds would be distributed twice.

Claim (PostgreSQL).

A later allocation year cannot reassess or re-divide amounts already assigned.

Falsified by: Year $n + 1$ changes an assigned year- n amount.

Matters because: Members' fixed shares would be unstable.

Claim (integration).

A distribution never marks a paid service year unpaid or revokes coverage.

Falsified by: Coverage or charge status changes when a distribution is paid.

Matters because: Service and distribution are separate transactions by rule.

Exit: a full simulated year, including a nonrenewing member who is still paid.

15 Milestone 7: Run a Small Supported Pilot

Package and sign the macOS application and versioned server and CLI artifacts. Complete user-held recovery and trusted-device enrollment with explicit device revocation and no provider decryption authority. Provide onboarding for annual service, requests, refunds, simulated distributions, follow-up preferences, encryption, recovery, and device loss; scoped diagnostics export; support and abuse intake; deletion procedures; backup and restore; and release rollback. Backups and retained obligation records respect separate keys and access roles.

Recruit recipients who receive substantial unsolicited contact and senders who want to reach them, including guests if Milestone 3b has shipped. Nondeveloper testers use clean machines and the pilot guide without direct database repair.

Claim (integration).

Restore followed by reconciliation reproduces every contract, request, refund, allocation, payable, and inbox state.

Falsified by: Any obligation differs between the pre-backup and reconciled post-restore state.

Matters because: Restore is the operator's last line of defense.

Claim (integration).

A revoked device cannot authorize commands or decrypt new content.

Falsified by: A command signed by a revoked device is accepted, or new content is readable to it.

Matters because: Device loss is a routine event.

Exit: signed releases, restore and recovery records, resolved critical usability findings, and a written summary of pilot findings.

16 Milestone 8: Portability and Phoros

Deliver Windows adapters once the macOS path is stable. Expose a narrow FFI facade for identity, requests, decisions, encrypted inbox, billing and coverage, financial status, distributions, and devices, with bounded values, stable error codes, cancellation, and explicit ownership. Generate Swift and Kotlin bindings with mock platform adapters and run the same client-core stories through each. Add a live payment connector.

For Phoros, its enrollment ceremony adopts an existing cs-mail subject through verified account control, as the product identity model prescribes. Version the identity contract as a published interface once two products consume it. Design cross-product account linking on explicit, granular, revocable consent.

17 Deferred and Open

- Bank-account replacement, full account management, and detailed erasure policy.
- Independent-provider federation and full SMTP relay operations.
- A full web inbox for members.

18 How an Implementation Agent Should Proceed

For each slice, read the applicable contracts, the Rust-domain principles, and local instructions. Name one user-visible outcome, identify its trust and persistence boundaries, agree the testing scope,

then implement contracts, application orchestration, adapters, and interface in that order. Report what changed, what was demonstrated, which claims were challenged, and what remains open or untested.

Do not bypass signed execution, write canonical tables from a CLI, persist plaintext for convenience, hide an uncertain charge, add tests merely to accompany code, or weaken an existing check to finish a milestone. A domain change requires an explicit decision and a reviewed cutover. Scope does not expand to live value, federation, or a full SMTP relay because an adapter exists.

If reconciliation fails, preserve the affected evidence and investigate the specific obligation. If secrets leak, stop accumulating affected data until the boundary is repaired. If a policy parameter is unresolved, use an explicitly versioned simulation policy and keep the live deployment gate closed.

19 First Assignment

Milestone 0, followed by the smallest Milestone 1 path: start the service, establish authenticated administration, run identity enrollment against the development OIDC provider, and expose one durable simulated annual charge. Later milestones extend that path without reimplementing domain rules.